TP2 — Gestion avancée des utilisateurs et des groupes

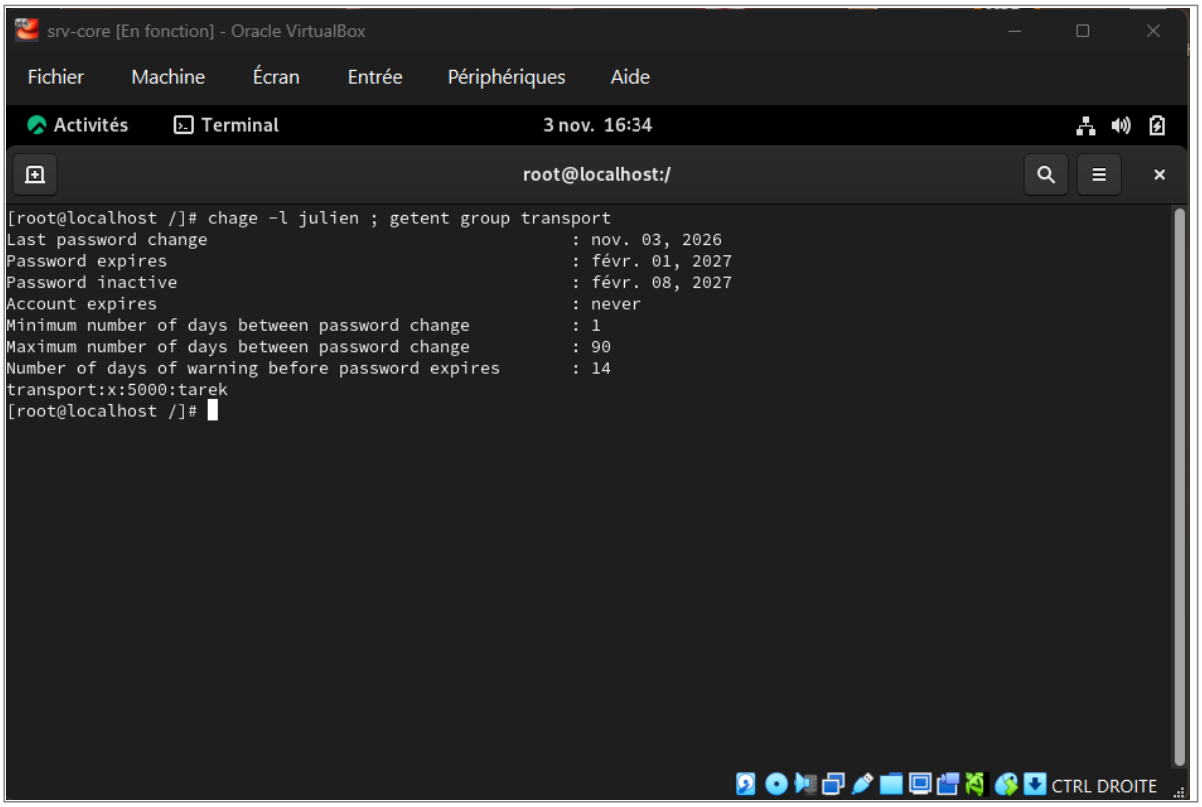
Mission 1 — Comptes, groupes et politique d'authentification

1.1 Politique AVANT la création des comptes

Un compte hérite de la politique en vigueur au moment de sa création : je la règle donc en premier. Dans `/etc/login.defs` : mot de passe valable 90 jours, délai minimal de 1 jour, avertissement 14 jours, umask de base `027`. Avec `pam_pwquality` : 12 caractères minimum et 3 classes de caractères. Avec `pam_faillock` : verrouillage après 5 échecs, déblocage automatique au bout de 600 s. Les fichiers correspondants sont fournis dans `configs/`.

1.2 Groupes et comptes

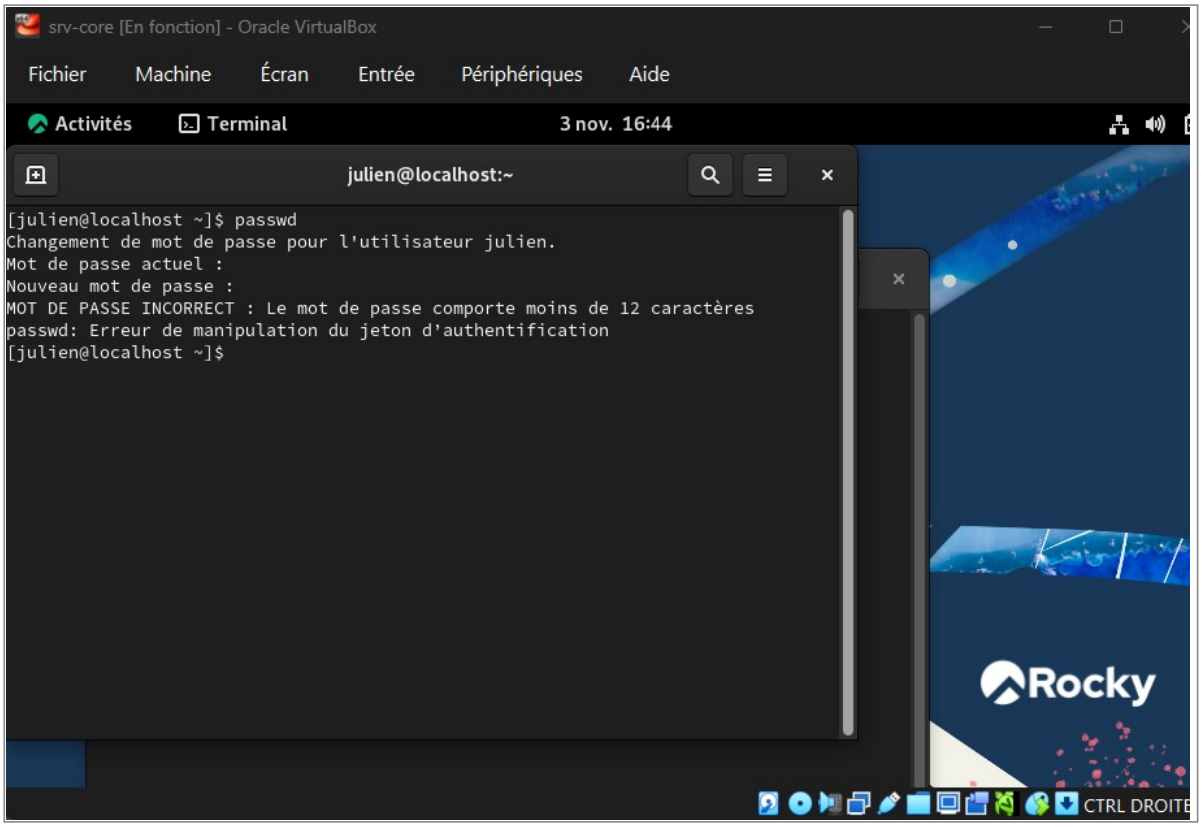
Je crée les groupes `transport` (5000), `telemetry` (5001) et `helios-ops` (5002), puis les comptes `julien`, `nadia`, `tarek` et `audit-ext`. Ce dernier expire 21 jours après sa création (mandat de l'auditeur externe).



`chage -l julien` confirme les règles : maximum 90 jours (expiration le 1er février 2027), minimum 1 jour, avertissement 14 jours, inactivité 7 jours. `getent group transport` n'affiche que `tarek` : `julien` et `nadia` ont `transport` pour groupe primaire, et un groupe primaire n'est pas listé dans les membres de `/etc/group`.

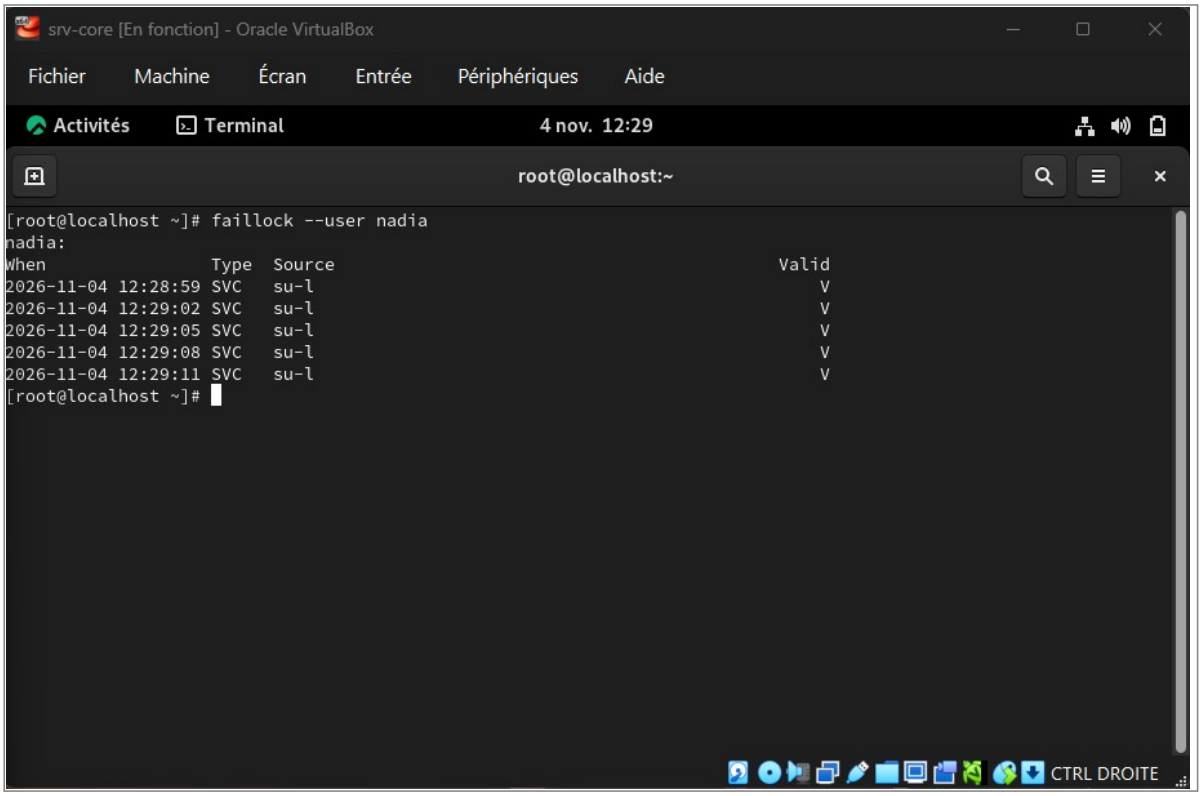
1.3 Qualité des mots de passe et verrouillage

Je teste la qualité des mots de passe : j'antidate d'un jour le dernier changement de julien (à cause de `PASS_MIN_DAYS=1`), puis je tente `passwd` avec le mot de passe `helios`.

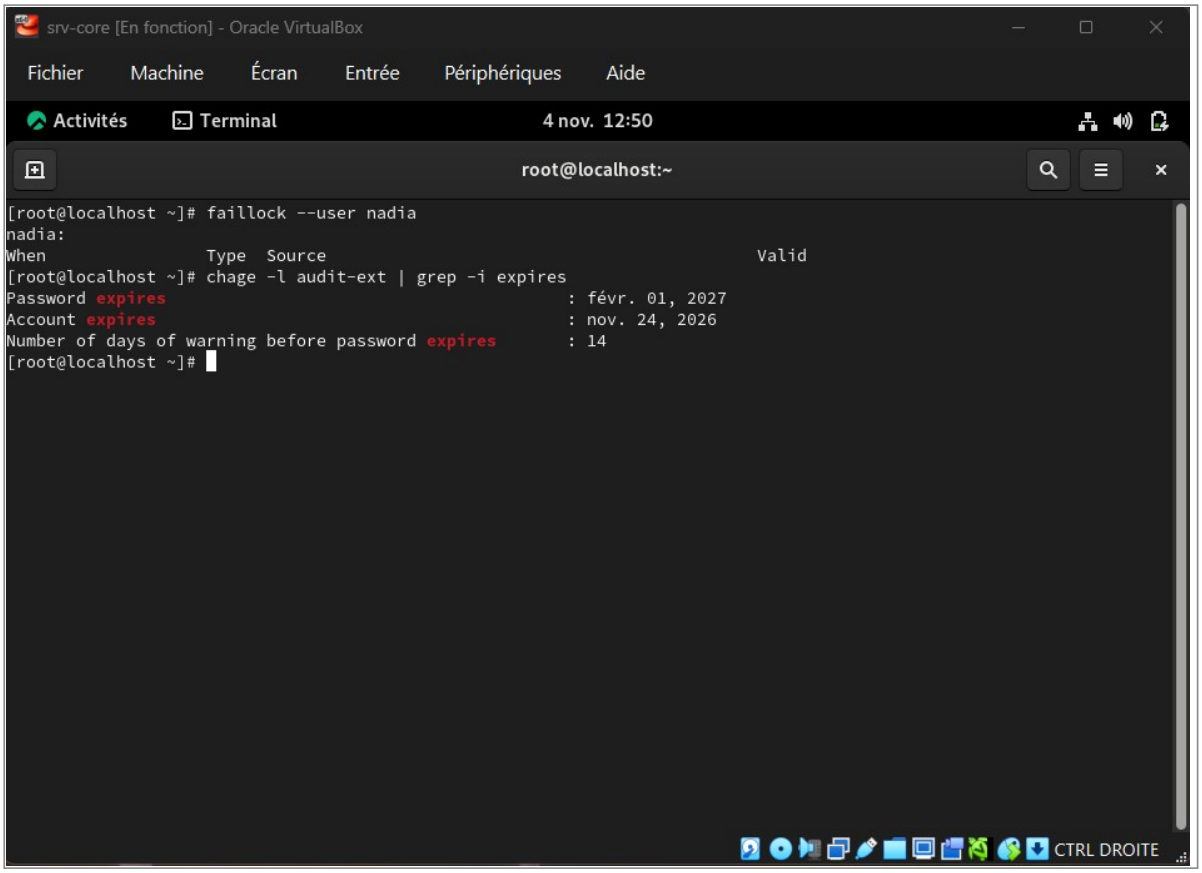


`pam_pwquality` refuse le mot de passe : « Le mot de passe comporte moins de 12 caractères ». `passwd` se termine par une erreur de manipulation du jeton d'authentification : le mot de passe de julien reste inchangé.

Pour le verrouillage, depuis un compte non root (`tarek`), j'enchaîne des échecs d'authentification sur `nadia` avec `su`.



`faillock --user nadia` liste 5 échecs (source `su-l`, tous valides : `V`), ce qui correspond à `deny = 5`.

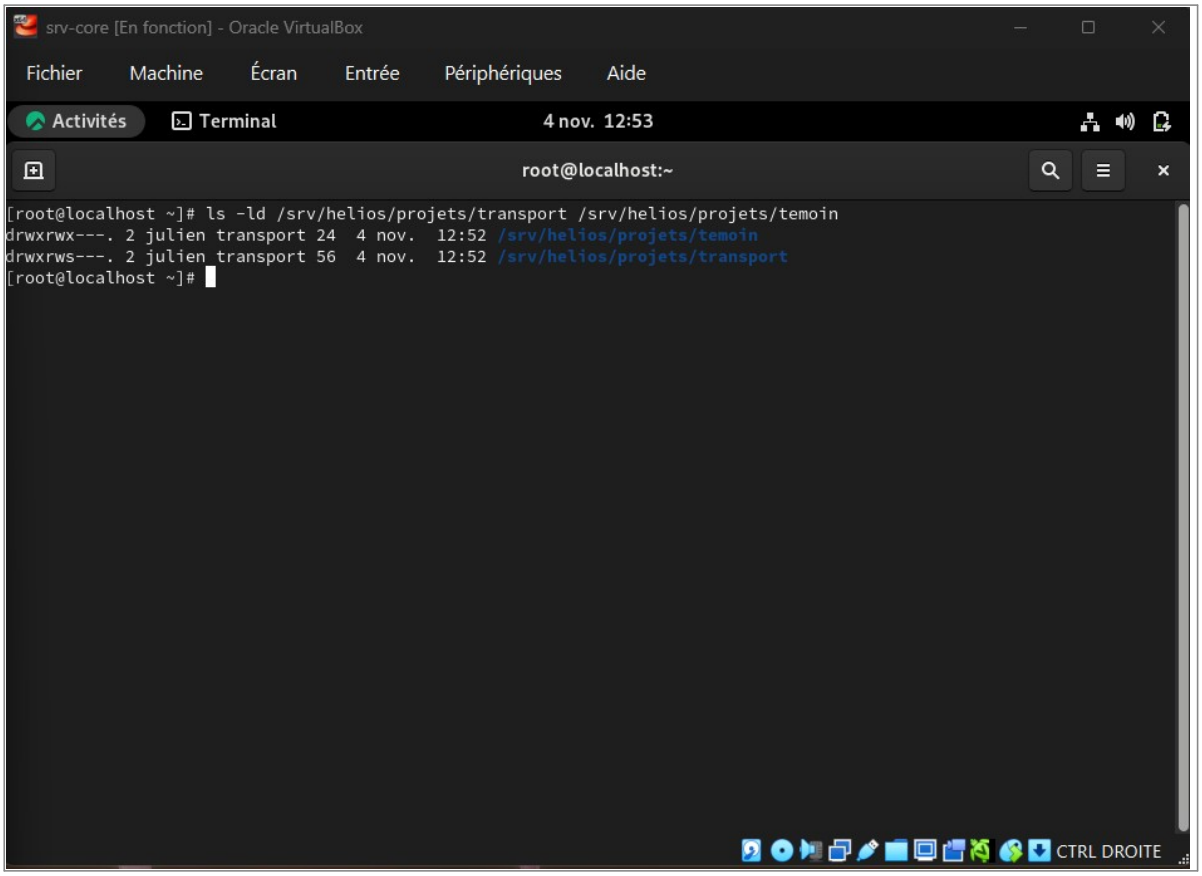


Après `faillock --user nadia --reset`, la liste est vide : le compte est déverrouillé. `chage -l audit-ext` indique `Account expires : nov. 24, 2026`, soit 21 jours après la création du compte.

Mission 2 — Bits spéciaux et umask

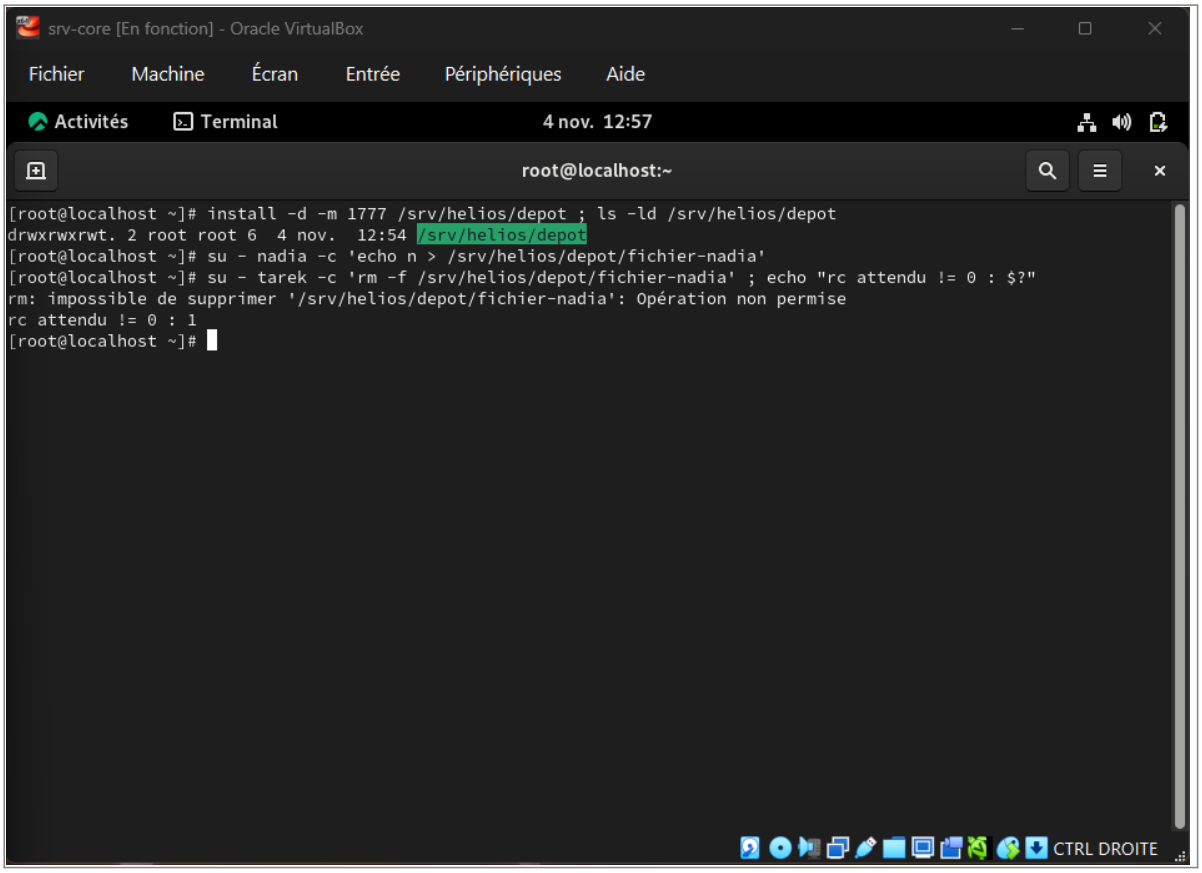
2.1 Répertoire setgid et dépôt sticky

Je crée deux répertoires `julien:transport` : `transport/` en `2770` (avec setgid) et `temoin/` en `0770` (sans setgid, répertoire témoin).



Le `s` à la place du `x` du groupe (`drwxrws---`) montre le setgid sur `transport/`, alors que `temoin/` reste en `drwxrwx---`. Avec le setgid, un fichier créé dans `transport/` reçoit le groupe du répertoire, quel que soit le groupe primaire de son créateur. Dans `temoin/`, il reçoit le groupe primaire du créateur.

Pour le dépôt commun `/srv/helios/depot`, je pose le mode `1777`. `nadia` y dépose un fichier, puis `tarek` essaie de le supprimer.

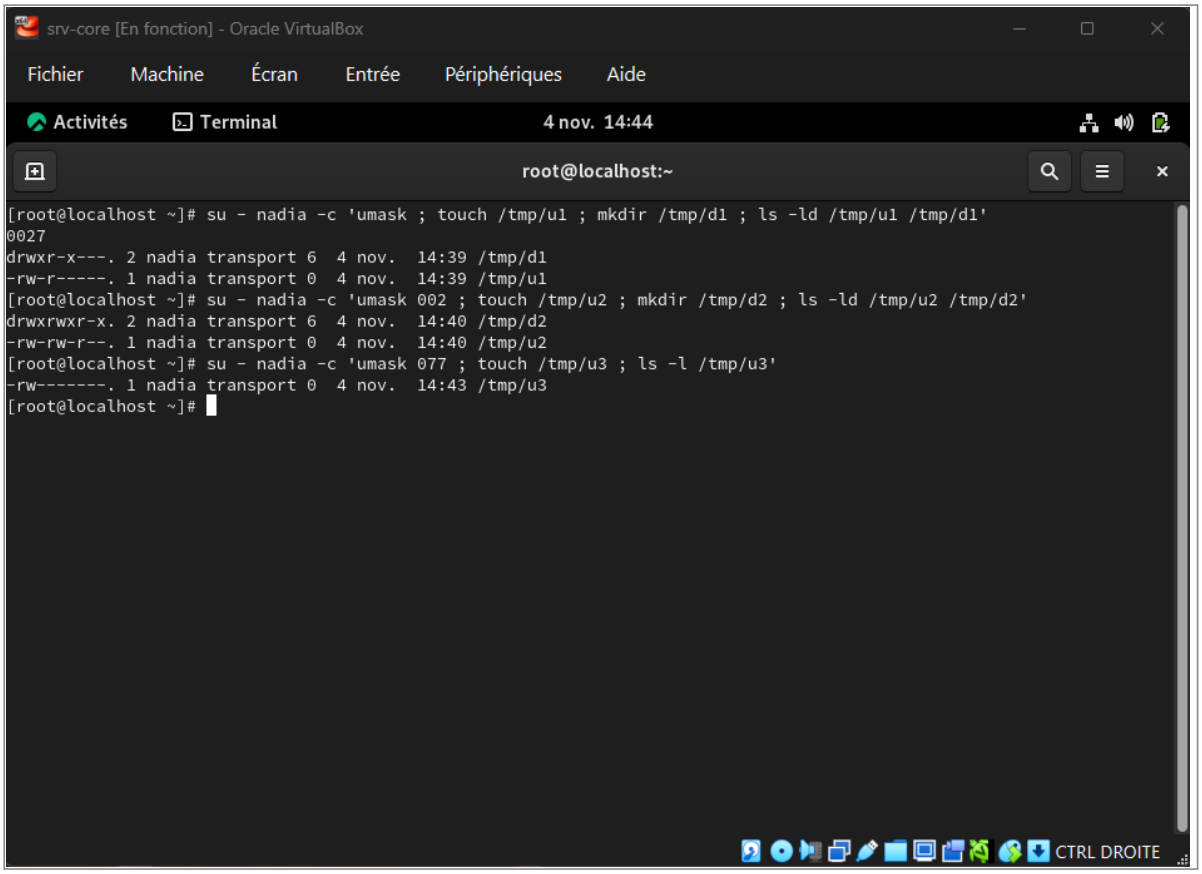


Le `t` final (`drwxrwxrwt`) indique le sticky bit. `rm` refuse la suppression (« Opération non permise », code retour 1) : même si tout le monde peut écrire dans le répertoire, seul le propriétaire du fichier, celui du répertoire ou root peut supprimer un fichier.

2.2 umask : prédire avant de mesurer

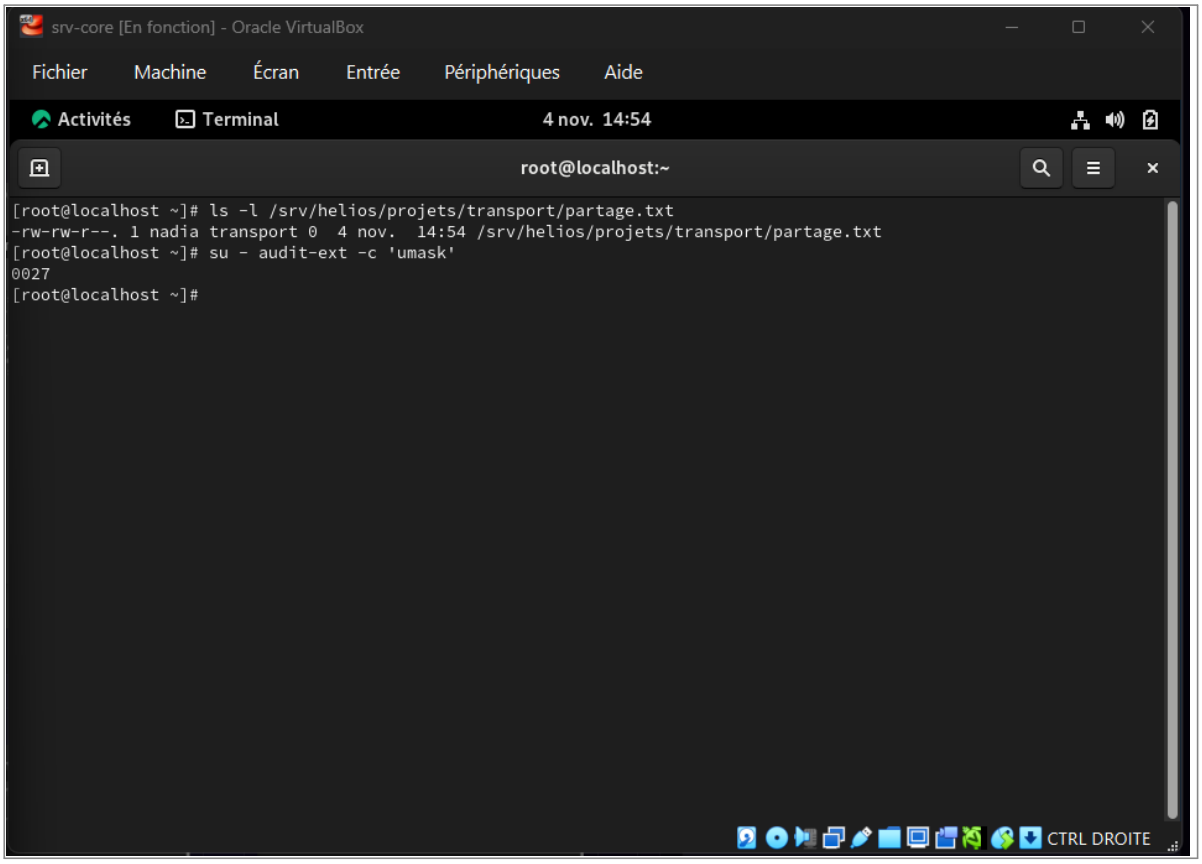
Un fichier part du mode 666 et un répertoire de 777, auquel on retire l'umask. Je note la prédiction avant de mesurer.

- **umask par défaut de nadia** — umask : 027 ; Fichier prédit : 640 ; Fichier observé : `-rw-r-----` ; Dossier prédit : 750 ; Dossier observé : `drwxr-x---`
- `umask 002` — umask : 002 ; Fichier prédit : 664 ; Fichier observé : `-rw-rw-r--` ; Dossier prédit : 775 ; Dossier observé : `drwxrwxr-x`
- `umask 077` — umask : 077 ; Fichier prédit : 600 ; Fichier observé : `-rw-------` ; Dossier prédit : — ; Dossier observé : — (non testé)



Les trois mesures correspondent aux prédictions. Le umask par défaut de nadia est `0027` : il vient de `UMASK 027` dans `login.defs`.

Je pose ensuite un umask collaboratif par groupe avec `/etc/profile.d/helios-umask.sh` : `002` pour les membres de `transport`, `027` pour les autres.



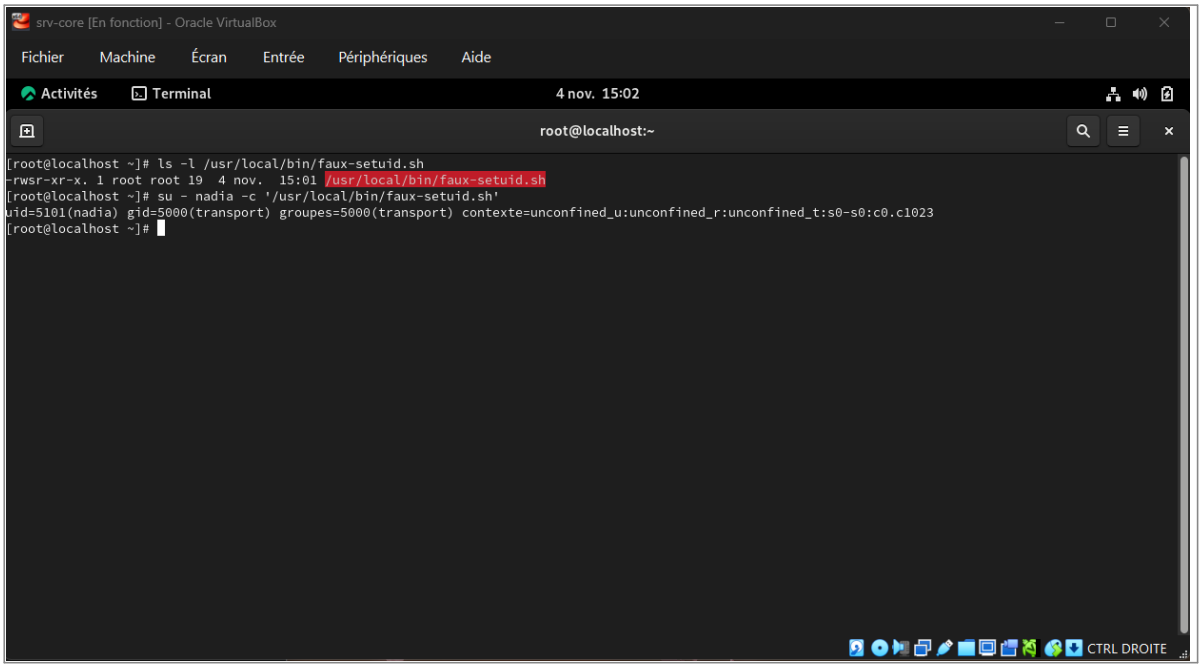
Un fichier créé par nadia dans `transport/` est en `-rw-rw-r--` (umask `002`) avec le groupe `transport`. `audit-ext`, qui n'est pas membre de `transport`, garde `0027`.

2.3 setuid : inventaire, échec, puis usage légitime

Trois binaires setuid courants et la raison de leur bit spécial :

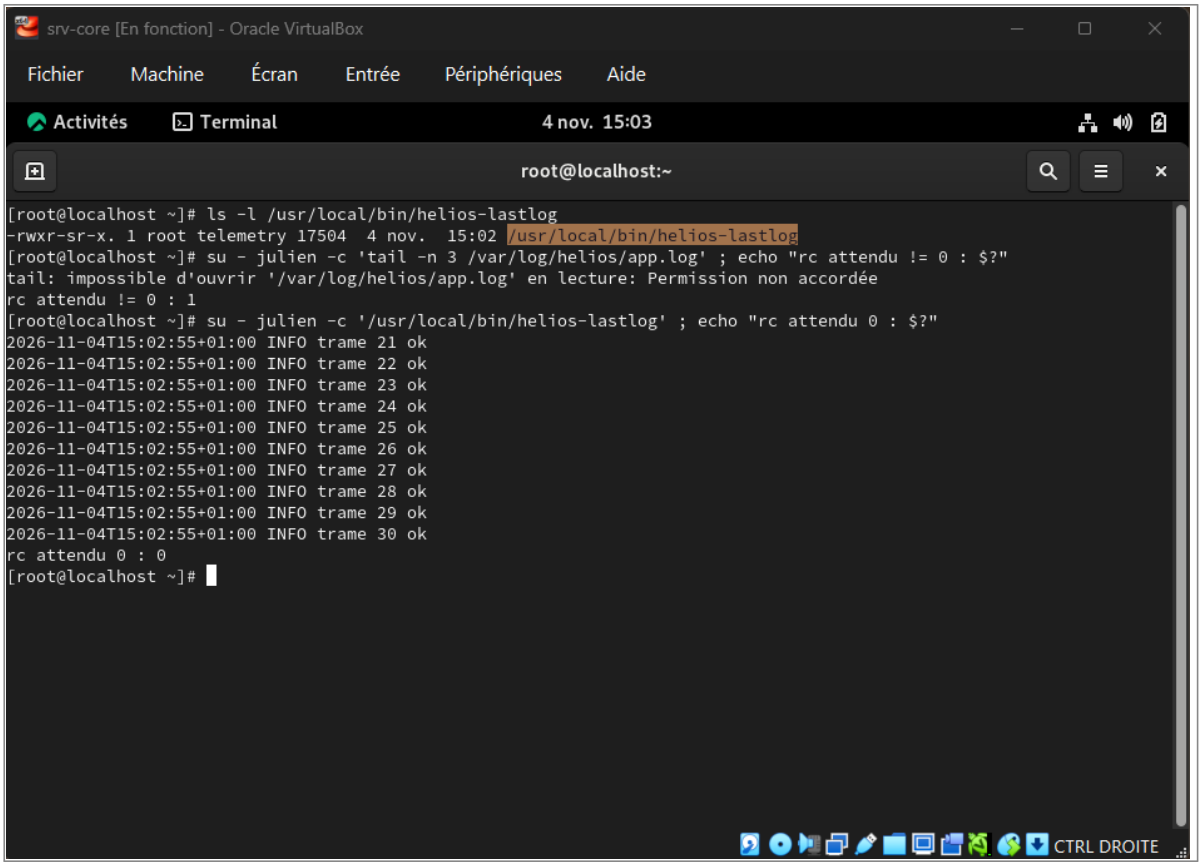
- `/usr/bin/passwd` — Justification : Doit écrire dans `/etc/shadow`, réservé à root, pour que chaque utilisateur puisse changer son propre mot de passe.
- `/usr/bin/sudo` — Justification : Doit prendre l'identité d'un autre utilisateur (souvent root) après avoir lu `sudoers` et authentifié l'appelant.
- `/usr/bin/mount` — Justification : Permet à un utilisateur ordinaire de monter les systèmes de fichiers autorisés, ce qui demande des privilèges.

Je place le bit setuid (`4755`) sur un script shell qui affiche `id`, puis je l'exécute en tant que `nadia`.



Le script est bien `-rwsr-xr-x root root`, mais il renvoie `uid=5101(nadia)` : le noyau Linux ignore le bit setuid sur un script interprété. Le script est supprimé ensuite.

Cas légitime : le journal `/var/log/helios/app.log` est réservé au groupe `telemetry`. Je compile `helios-lastlog`, un binaire `root:telemetry` en `2755` qui lance `tail -n 10` avec un chemin et un environnement figés.

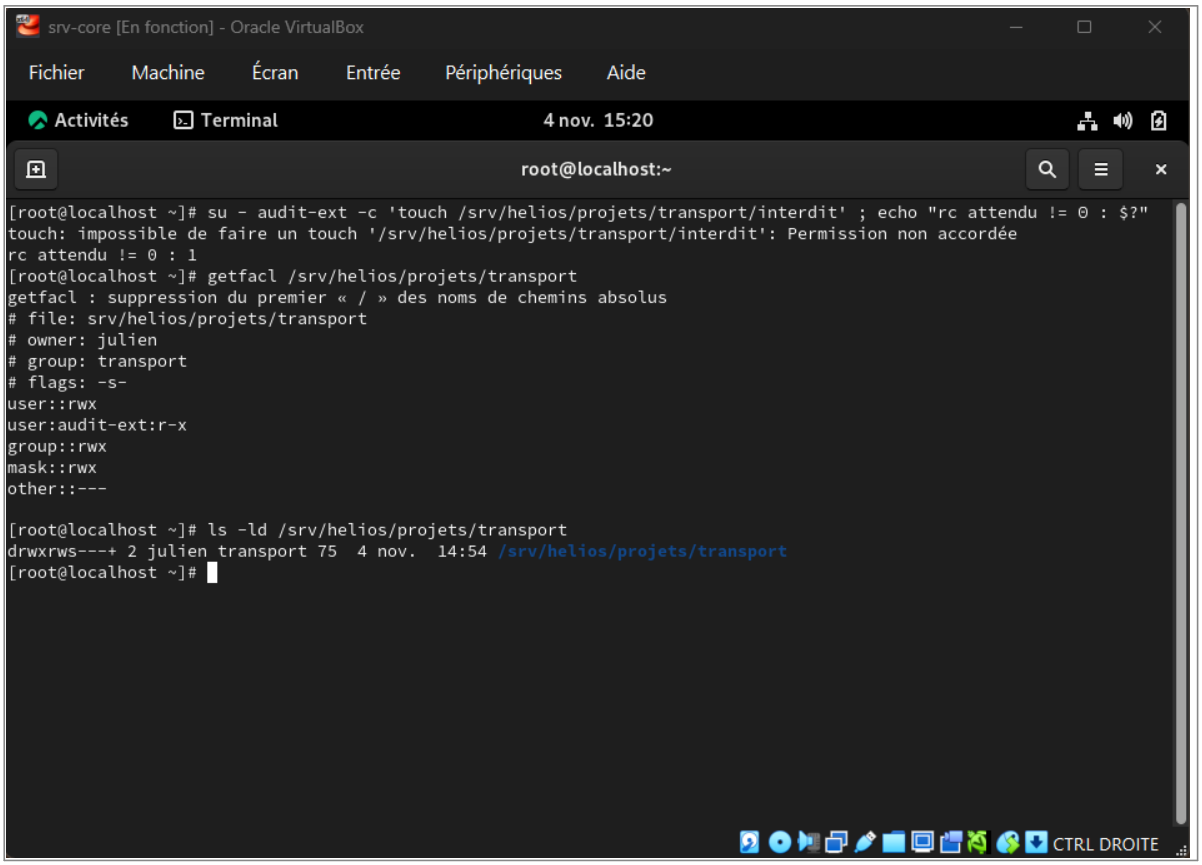


Le binaire est `-rwxr-sr-x`. `julien` ne peut pas lire le journal avec `tail` directement (« Permission non accordée », code 1), mais il le lit avec `helios-lastlog` (code 0, 10 lignes). L'egid `telemetry` donne la lecture, l'uid reste celui de julien : c'est préférable à un setuid root, car les privilèges sont limités à un groupe.

Mission 3 — ACL POSIX

3.1 Accès nominatif hors du groupe

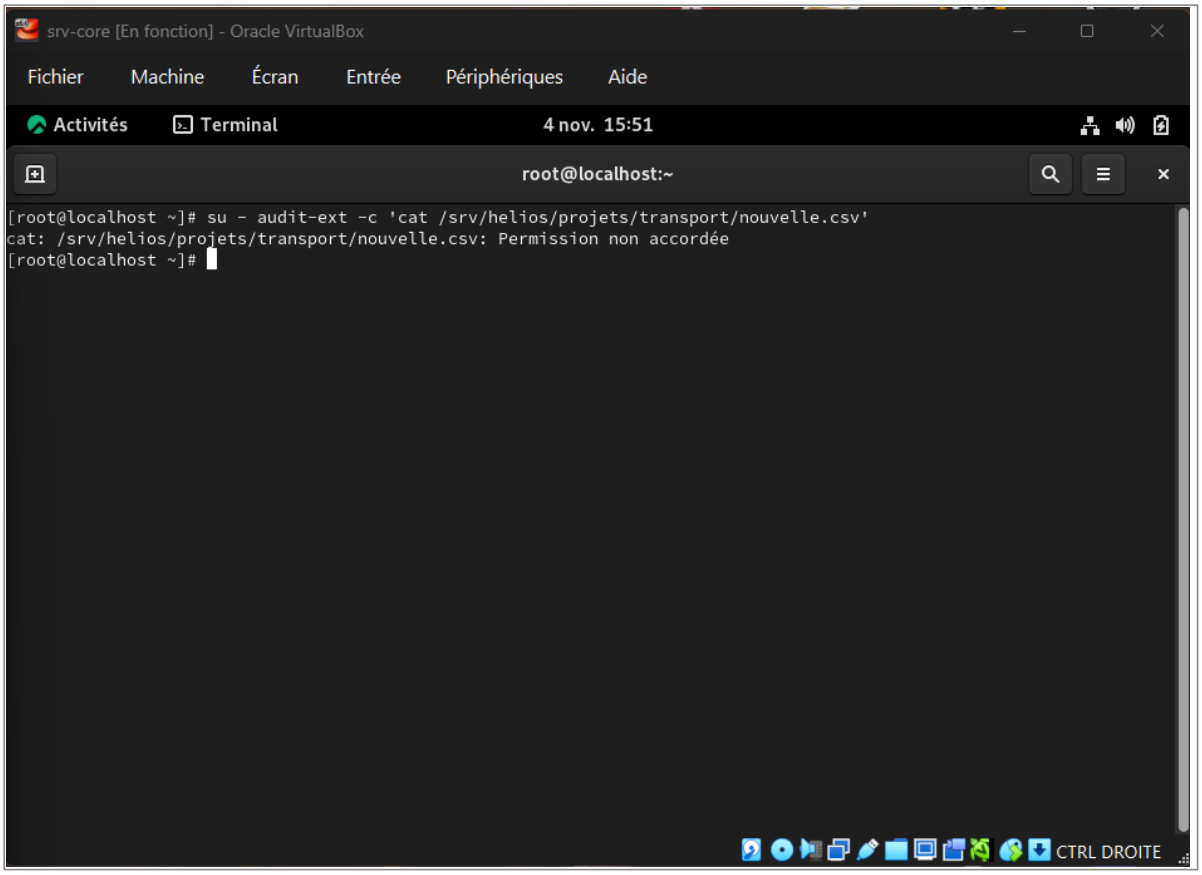
J'ajoute l'ACL `u:audit-ext:rX` sur `transport/` (avec `-R`) : le `X` majuscule n'ajoute `x` que sur les répertoires.



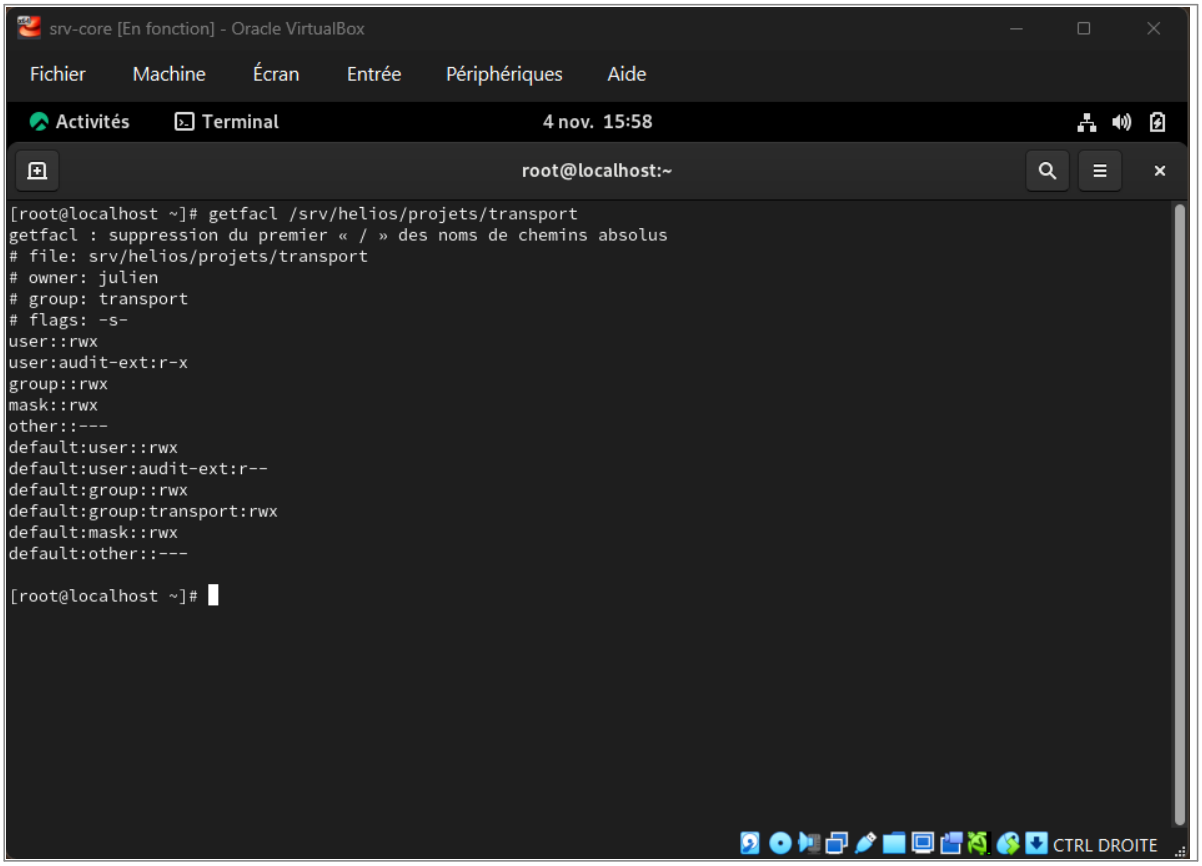
`getfacl` montre `user:audit-ext:r-x` et `ls -ld` affiche un `+` après les droits, signe de la présence d'ACL. Le `touch` d'audit-ext est refusé (code 1) : l'auditeur peut lire sans écrire, sans être membre du groupe `transport`.

3.2 ACL par défaut

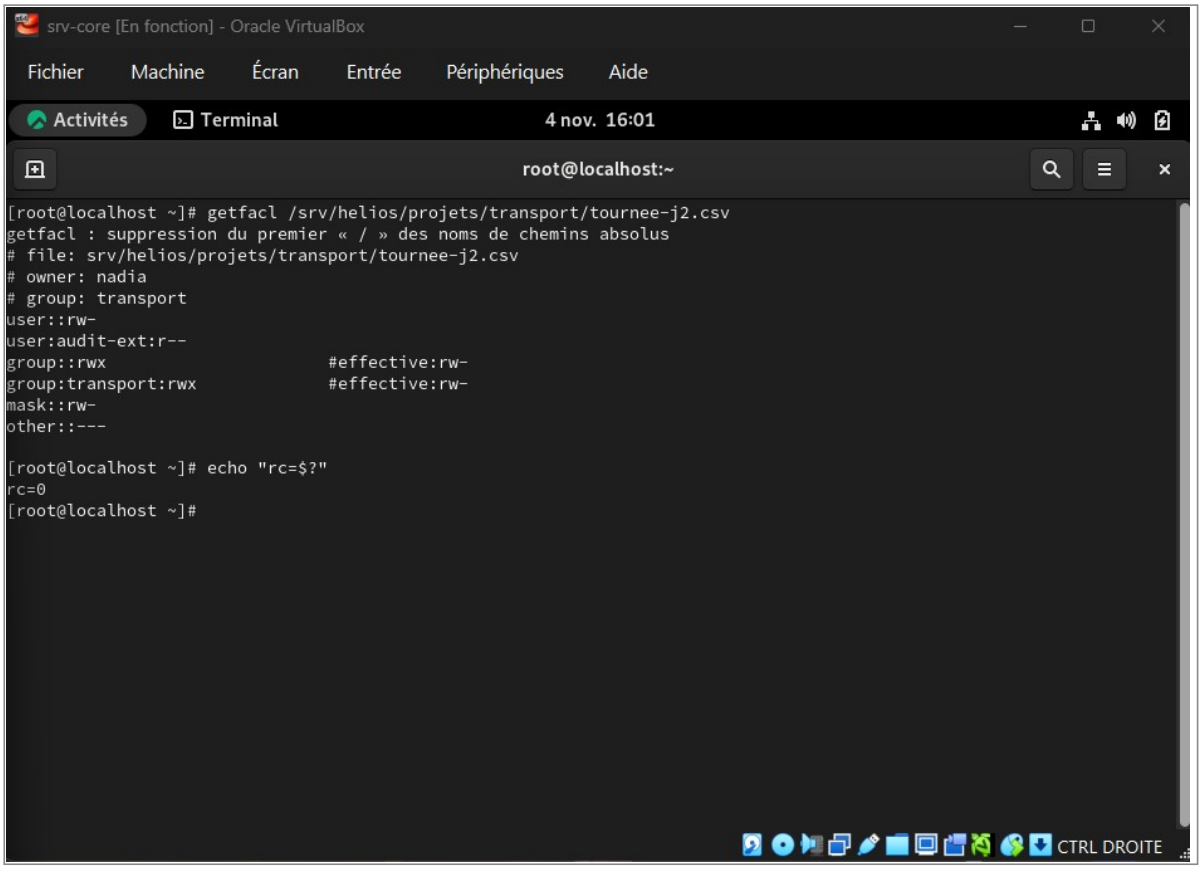
Je constate d'abord le problème : un fichier créé par nadia après la pose de l'ACL n'est pas couvert, car `-R` ne s'applique qu'aux fichiers existants.



`audit-ext` n'arrive pas à lire `nouvelle.csv` (« Permission non accordée »). Pour corriger, je pose des ACL par défaut : `d:u:audit-ext:r`, `d:g:transport:rwx` et `d:o::---`.



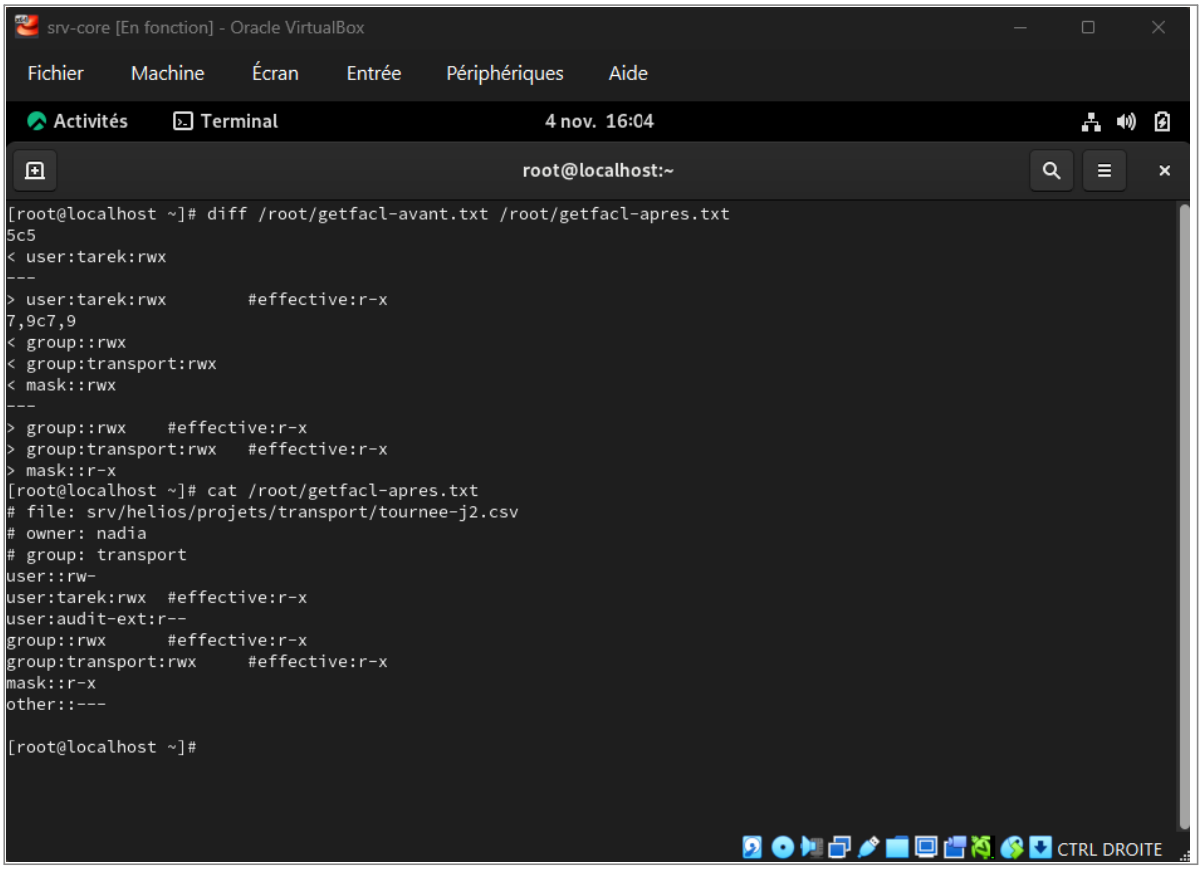
Les lignes `default:` apparaissent : tout nouveau fichier ou sous-répertoire hérite de ces entrées.



`tournee-j2.csv`, créé par nadia après la pose des ACL par défaut, porte `user:audit-ext:r--` sans aucune action manuelle, et la lecture par audit-ext réussit (`rc=0`). Le masque est `rw-` parce que le fichier est créé en `664`, sans `x`.

3.3 Le piège du masque effectif

J'ajoute `u:tarek:rwx` sur `tournee-j2.csv`, j'enregistre `getfacl` (avant), je lance `chmod g-w`, puis je compare (après).

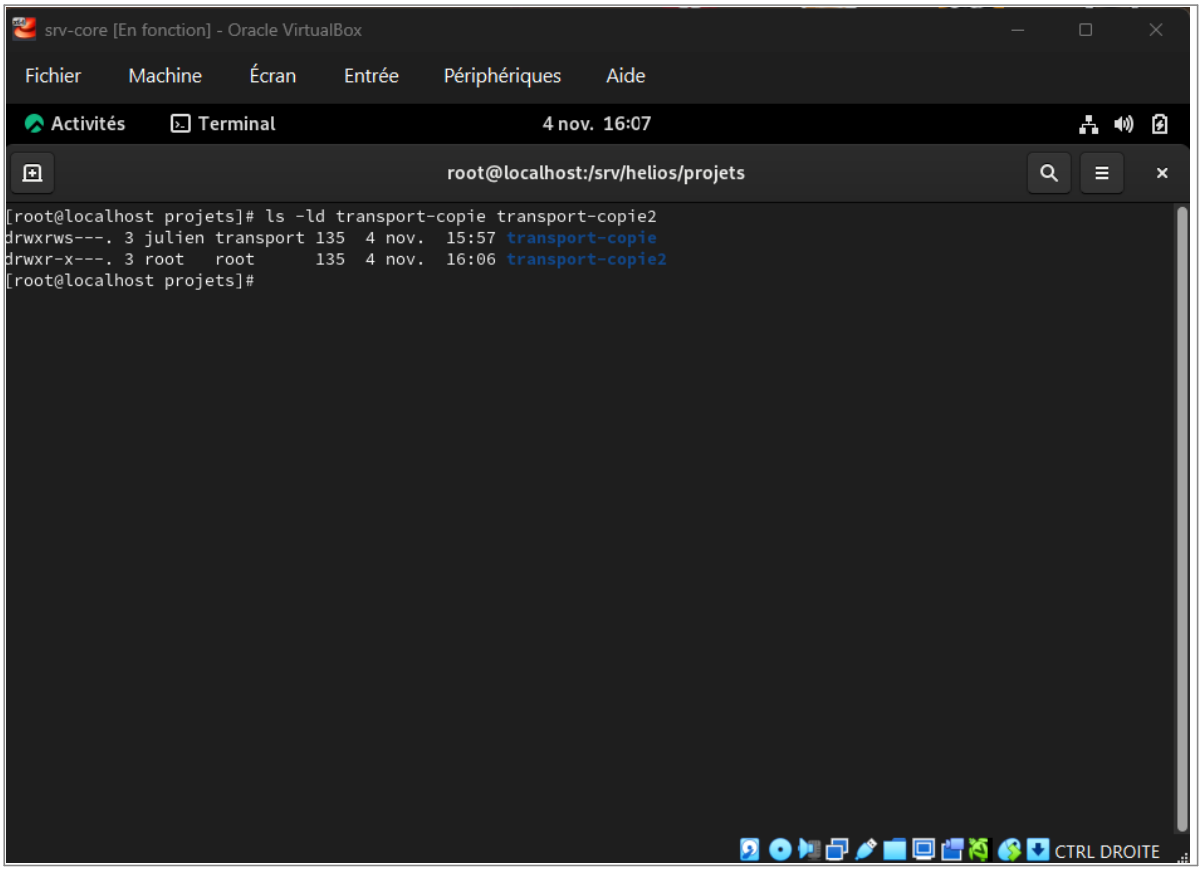


Le `diff` montre que `mask::rwx` est devenu `mask::r-x` : les lignes `user:tarek`, `group::` et `group:transport` passent à `#effective:r-x`.

Ce que `chmod g-w` a réellement modifié. Sur un fichier qui porte des ACL, les bits « groupe » de `chmod` ne changent plus le groupe propriétaire : ils changent le **masque** (`mask::`). Retirer `w` du masque plafonne les droits effectifs de tous les utilisateurs et groupes nommés (tarek, `group::`, `group:transport`). Leurs entrées ne changent pas, mais leur écriture est bloquée. `setfacl -m m::rwx` rend le masque, donc les droits effectifs, à tarek.

3.4 Sauvegarder, restaurer, transporter les ACL

Je sauvegarde les ACL avec `getfacl -R transport > acl-transport-<date>.txt`, puis je compare `cp -a` et `cp -r`.



`cp -a` conserve le propriétaire, le groupe et le mode du répertoire (`julien:transport`, `drwxrws---`), alors que `cp -r` produit `root:root` en `drwxr-xr-x`. `tar --acls --xattrs` conserve aussi les ACL dans une archive.